FLUTTER · CLEAN ARCHITECTURE · DATA LAYER

TaskFlow — Senior Exam · Days 1–6

100 MCQs Single best answer ~90–120 min Closed book

> Choose the single best answer for each question. Answer positions are balanced and shuffled — there is no positional pattern to exploit. Full answer key with rationale is in the companion document.

Day 1 — Clean Architecture & the Dependency Rule

1. code · medium · Analyze

A junior pushes this file, located at `domain/repositories/task_repository.dart`. The CI architecture-lint fails. Beyond "it imports Dio," what is the **deepest** reason this is wrong?

```
import 'package:dio/dio.dart';
import '../entities/task.dart';

abstract interface class TaskRepository {
  Future<Response> getTasks({String? cursor});
}
```

A. The contract returns `Response` instead of `Either<Failure, T>`, so it cannot represent failures as values.

B. `abstract interface class` is invalid for a repository; only `abstract class` may be implemented across layers.

C. The domain's contract now depends on a network library, so the business rules can no longer be swapped onto a non-Dio transport — the abstraction is no longer an abstraction.

D. The import path uses `../entities/task.dart` instead of an absolute package import, breaking the dependency rule.

2. scenario hard Evaluate

Sara's team must switch from REST to GraphQL. The domain is pure, contracts live in `domain/`, mapping/error-conversion happen in `data/`. A teammate claims: "We'll still have to touch the domain, because the use cases call `getTasks()` and the data source is changing." Correct assessment?

A. Wrong — use cases depend only on the `TaskRepository` abstraction; the GraphQL change is absorbed in the data source, mapper, and impl, leaving domain untouched.
B. Correct — entities mirror the API payload, so a GraphQL schema change reshapes the `Task` entity and ripples into domain.
C. Correct — use cases reference the data source signature, so changing the transport forces a use-case edit.
D. Wrong — but only because GraphQL and REST return identical JSON, so no mapper change is needed either.

3. conceptual medium Understand

At compile time, `data` imports `domain`; yet at runtime a domain use case ends up causing the data implementation to run. Which statement best explains this apparent contradiction?

A. The dependency arrow and the control-flow arrow point the same way; the use case imports the impl lazily to avoid a cycle.
B. Reflection resolves the implementation at runtime, so no compile-time dependency exists in either direction.
C. Domain imports the data implementation only inside test files, so production has no cycle.
D. The compile-time dependency points inward (data→domain) while the runtime control flows outward (domain→data) via an injected implementation of the contract.

4. code hard Analyze

This `presentation/providers/task_providers.dart` compiles and the list renders. Why is it a serious architectural defect despite working?

```
final tasksProvider = FutureProvider((ref) async {
  final dtos = await ref.read(remoteDataSourceProvider).getTasks();
  return dtos; // List<TaskDto>
});
```

A. Providers may not be `FutureProvider`; list data must use `StreamProvider` to stay reactive.
B. The presentation layer reaches past the use case and contract straight to the data source, and exposes DTOs to the UI — coupling widgets to the JSON shape and bypassing error-to-Failure conversion.
C. `ref.read` is used instead of `ref.watch`, so the UI will never rebuild when tasks change.
D. Returning a `List<TaskDto>` is fine, but the provider should be `autoDispose` to avoid a memory leak.

5. scenario · medium · Apply

A reviewer finds the abstract `task_repository.dart` sitting in `data/repositories/` next to its impl. It compiles and tests pass. What real harm does this misplacement cause, even though nothing is broken yet?

A. Putting the contract in `data/` forces the domain (which uses the contract) to import the data layer, inverting the dependency rule back the wrong way.

B. The domain can no longer reference its own contract, so use cases fail to compile.

C. None of consequence — folder location is a cosmetic convention; the dependency rule is about imports, not paths.

D. It causes a runtime dependency-injection cycle because the impl and the interface share a directory.

6. code · medium · Analyze

Given the real `Task` entity (`extends Equatable`, fields `id/title/isDone/dueDate`), a teammate proposes adding `factory Task.fromJson(...)` directly onto it "to save writing a separate DTO." Why does a senior reject this?

A. `Equatable` classes cannot have factory constructors, so it won't compile.

B. It's acceptable as long as `dueDate` is parsed with `DateTime.parse`, since that's part of Dart core.

C. `fromJson` must be `static`, not a `factory`, so the proposal is merely a syntax error.

D. It would pull JSON-parsing knowledge into the domain entity, coupling pure business shape to the API's wire format and erasing the DTO boundary.

7. scenario · hard · Evaluate

For a 3-screen internal tool with a 2-week lifespan, a contractor builds full Clean Architecture; the client complains it took twice as long. A junior concludes "Clean Architecture is always over-engineering." The senior's nuanced position?

A. The junior is right; Clean Architecture is for large apps only and should never be used under 5 screens.

B. The contractor is right; skipping any layer is unprofessional regardless of app lifespan.

C. Both miss the point: the cost is real and was poorly matched to a throwaway app, but the same boundaries pay off the first time a long-lived, team-maintained app's requirements change — it's a lifespan/team trade-off, not a dogma.

D. The real issue was using DTOs and mappers; contracts and use cases are free, so only the data-layer indirection was wasteful.

8. code hard Analyze

In the real `TaskRepositoryImpl.getTasks`, what two distinct boundary responsibilities is this single method discharging?

```
try {
  final dtos = await _remote.getTasks(cursor: cursor);
  return Right(dtos.map((d) => d.toEntity()).toList());
} on UnauthorizedException {
  return const Left(UnauthorizedFailure());
} on NetworkException {
  return const Left(NetworkFailure());
}
```

A. It converts DTOs to entities (so the API shape never escapes) and converts exceptions to typed `Failure` values (so raw errors never escape into the domain/UI).

B. It validates business rules and persists results to the local cache.

C. It maps entities to DTOs for the outgoing request and rethrows exceptions for the provider to catch.

D. It performs dependency injection of the data source and unwraps the `Either` for the use case.

9. conceptual medium Apply

Why is returning `Either<Failure, List<Task>>` from the repository considered superior to letting it `throw NetworkException` for the use case to catch?

A. `Either` is faster at runtime because it avoids the cost of building stack traces.

B. Throwing forces every caller to remember a try/catch and lets raw, untyped exceptions leak across the boundary; `Either` makes failure an explicit, typed value the caller must handle in the type system.

C. `Either` allows the repository to return multiple successful values at once, which exceptions cannot.

D. Exceptions cannot cross an `async` boundary in Dart, so `Either` is technically required here.

10. scenario medium Analyze

A team organizes `lib/` as `lib/models/`, `lib/screens/`, `lib/services/` (layer-first, app-wide). Six months in, Tasks and Billing are hard to work on in parallel and a deleted feature leaves dead files scattered everywhere. What does the lesson prescribe, and why?

A. Keep layer-first but add a `lib/core/` to hold shared services, which resolves parallel-work conflicts.

B. Move everything into `domain/` since that is the only layer guaranteed to be stable.

C. Merge `models` and `services` into one folder so each feature has fewer files to coordinate.

D. Switch to feature-first (`features/<feature>/{data,domain,presentation}`), so each feature is a self-contained vertical slice that can be built, deleted, or owned independently.

11. code hard Evaluate

A developer defends a `domain/entities/task.dart` that does `import 'package:flutter/material.dart';` and holds a `Color labelColor` field: "It's only using `Color` as a data type — no widgets, so the dependency rule is fine." Evaluate.

A. Invalid — importing anything from `package:flutter` makes the domain depend on the UI framework; "it's just a type" still binds business code to Flutter and breaks portability/testability.

B. Valid — `Color` lives in `dart:ui`, which is core Dart, so no Flutter dependency is introduced.

C. Valid — the rule only forbids importing widgets and build methods; plain types like `Color` are exempt.

D. Invalid only because `Task` no longer extends `Equatable`; once that's restored the Flutter import is acceptable.

12. scenario hard Evaluate

To unit-test `GetTasks` without a server, a junior imports `TaskRepositoryImpl` and feeds it a mocked `Dio`. A senior says "you've made the domain test depend on the data layer." Better approach and why?

A. Keep using `TaskRepositoryImpl` but point Dio at a localhost mock server — closer to production behavior.

B. Move the use-case test into the data layer's test folder, since that's where the repository implementation lives.

C. Provide a fake/mock that implements the `TaskRepository` contract directly, so the use-case test depends only on the domain abstraction and needs no Dio, network, or data-layer code.

D. Make `GetTasks` accept a `Dio` instance so tests can inject a mock client without a repository at all.

13. conceptual medium Understand

The backend renames JSON field `due_date` to `deadline`. A correct architecture localizes the change to what, and what stays untouched?

A. Change ripples to the entity, use cases, and any widget showing the date; the mapper is bypassed for renames.

B. Change is localized to the DTO and the mapper; the `Task` entity, use cases, and UI stay untouched.

C. Change is localized to the use case, which re-reads the field name; the DTO and entity are unaffected.

D. Change requires updating the repository contract's method signature so callers know the field moved.

14. code · medium · Analyze

Both the real contract and impl begin with `import 'package:fpdart/fpdart.dart' hide Task;`. Given fpdart exports a `Task` type, what does `hide Task` prevent, and what happens without it?

A. It avoids a name collision between fpdart's `Task` and the domain `Task` entity; without it, references to `Task` would be ambiguous and fail to resolve.

B. It hides the domain `Task` from fpdart; without it, the entity would be invisible to the repository.

C. It is purely stylistic; without it the code compiles identically.

D. It prevents fpdart from overriding `Either`, which shares a name with the domain layer.

15. scenario · hard · Analyze

Omar stores the token in a global variable and parses JSON inside widgets. Setting aside style, what is the **precise architectural reason** the "switch to GraphQL + add offline" request detonated his codebase while Sara swapped one folder?

A. Omar used `setState` instead of Riverpod, so state couldn't survive the data-source change.

B. Omar's problem was purely the global token; once moved to secure storage, the GraphQL switch would have been one edit.

C. Sara used a faster JSON library, so her parsing survived the GraphQL switch unchanged.

D. Omar had no boundaries: API/JSON knowledge was duplicated across many widgets, so the data concern had dozens of change-sites instead of one isolated, swappable layer.

16. conceptual · medium · Apply

Adding a `Project` feature: where do the `ProjectRepository` interface and its implementation each belong, and why?

A. Both interface and impl in `data/`, because repositories are a data concern and keeping them together simplifies wiring.

B. Interface in `presentation/` (the UI declares what it needs), impl in `data/`, so the UI drives the contract.

C. Interface in `domain/repositories/`, impl in `data/repositories/`, because the contract is a business promise the domain owns, and placing it inward makes data depend on domain (dependency inversion).

D. Interface in `domain/`, impl also in `domain/`, since keeping a contract near its implementation aids readability and the data layer only holds DTOs.

17. scenario hard Evaluate

A team adopts Clean Architecture but lets presentation call repository implementations directly and skips use cases "to reduce boilerplate." Months later they must add logging + a permission check to every task action. Where does the shortcut hurt most, and what does it reveal about use cases?

A. It hurts nowhere meaningful — use cases are ceremonial wrappers, so the change is just as easy at the provider level.

B. It hurts because cross-cutting business actions now have no single home; use cases existed precisely to be the one orchestration point per business action where such logic lands, instead of being copy-pasted across providers.

C. It hurts only performance, since calling the impl directly adds an extra object allocation per request.

D. It hurts because skipping use cases breaks the dependency rule, forcing the domain to import presentation.

Day 2 — Networking & the API Client

18. code hard Analyze

`buildDio()` registers interceptors in this order. A reviewer rejects it. What concrete, observable defect does this ordering produce?

```
dio.interceptors.add(LogInterceptor(requestBody: true));   // 1
dio.interceptors.add(AuthInterceptor(() => readToken()));  // 2
dio.interceptors.add(ErrorInterceptor());                  // 3
```

A. The request log is written before the `Authorization` header is added, so logs show requests missing the Bearer token even though it was actually sent.

B. The `ErrorInterceptor` will fire before the `AuthInterceptor` on the response path, so 401s are mapped before a token is ever attached.

C. `LogInterceptor` and `AuthInterceptor` both run `onRequest`, so the token gets added twice and the server rejects a malformed header.

D. Dio runs interceptors in reverse registration order on the request path, so auth fires first anyway and the ordering is harmless.

19. scenario · medium · Evaluate

On a flaky metro signal, a `GET /tasks` and a `POST /tasks` both fail with `receiveTimeout`. Your retry interceptor retries *any* request on timeout, twice. The senior objection?

A. The policy is correct — both GET and POST are idempotent by HTTP definition, so retries can never cause duplicates.

B. Retrying either is unsafe because timeouts always mean the server never received the request, so a retry just doubles load for nothing.

C. Retrying the GET is fine, but retrying the POST risks creating duplicate tasks because a receiveTimeout can occur *after* the server already processed and committed the write.

D. The GET retry is the dangerous one, because reading the same resource twice can return stale data and corrupt the local cache.

20. conceptual · medium · Understand

A request returns **403 Forbidden**. A junior wires the auth interceptor to trigger the refresh-token flow on 403 just like 401. Why is this wrong?

A. 403 means the token is expired, so refresh is correct, but the retry must use PUT instead of GET.

B. 403 means the user is authenticated but lacks permission for this resource; a fresh token grants no new permissions, so refresh-and-retry loops pointlessly.

C. 403 means the request body failed validation; you should resubmit with corrected fields, not refresh.

D. 403 means the URL is wrong; refreshing is harmless but you should also retry against a corrected path.

21. code hard Analyze

This refresh logic ships to production. The refresh endpoint itself starts returning 401 (its refresh token also expired). What happens?

```
void onError(DioException err, ErrorInterceptorHandler handler) async {
  if (err.response?.statusCode == 401) {
    final newToken = await _refresh();          // POST /auth/refresh
    final opts = err.requestOptions;
    opts.headers['Authorization'] = 'Bearer $newToken';
    handler.resolve(await _dio.fetch(opts));    // retry original
  } else {
    handler.next(err);
  }
}
```

A. Dio detects the duplicate URL and short-circuits, returning the original error after one extra call.

B. Because `_refresh()` is awaited, the second 401 is caught by the try/catch and the loop terminates after exactly two attempts.

C. `handler.resolve` swallows the second 401 silently, so the user simply sees an empty task list with no error.

D. The refresh call's own 401 re-enters this same `onError`, triggering another refresh, which 401s again — an infinite refresh loop until the stack or rate limiter gives out.

22. scenario medium Apply

Your access token expires mid-session while three screens fire GETs within the same 200 ms window; all three get 401. Your interceptor refreshes on each 401 without coordination. The senior-level problem and fix?

A. Three concurrent refreshes race; later ones may invalidate the token earlier ones just stored. Queue requests during an in-flight refresh so only one refresh runs and all three retry with the result.

B. No problem — three refreshes simply return the same token; idempotent and harmless.

C. The three requests should each be downgraded to 403 to avoid refreshing more than once per minute.

D. Refresh should be skipped entirely; the user should be logged out on the first 401 to keep state consistent.

23. conceptual easy Understand

What precisely is the difference between `connectTimeout` and `receiveTimeout`?

A. `connectTimeout` bounds uploading the request body; `receiveTimeout` bounds establishing the connection.

B. `connectTimeout` bounds the total round trip; `receiveTimeout` bounds only the DNS lookup phase.

C. `connectTimeout` bounds establishing the TCP/TLS connection; `receiveTimeout` bounds the wait for response data to arrive after the request is sent.

D. They are aliases — Dio uses whichever is smaller and ignores the other.

24. code medium Analyze

Each data source method constructs a fresh `Dio(...)` even though a fully configured shared `buildDio()` (auth, retry, error interceptors) exists. Beyond "duplication," the most damaging runtime consequence?

```
Future<List<TaskDto>> getTasks() async {
  final dio = Dio(BaseOptions(baseUrl: 'https://dummyjson.com'));
  final res = await dio.get('/todos');
  ...
}
```

A. Connection pooling forces all `Dio()` instances to share one socket, so the new instance steals the shared client's open connections.

B. Dart will throw at runtime because two `Dio` instances cannot target the same `baseUrl` concurrently.

C. The local `Dio` inherits the shared instance's interceptors by default, so behavior is identical and only memory is wasted.

D. Each call allocates a throwaway `Dio` with none of the shared interceptors, so requests go out with no token, no retry, and no error translation — silently bypassing all cross-cutting behavior.

25. scenario medium Apply

A teammate stores both tokens in `SharedPreferences` "because it's simpler and persists." On which threat does this concretely fail, and what's the right store?

A. It fails on app-update migrations because SharedPreferences is wiped on upgrade; use a database table instead.

B. It fails on rooted/jailbroken or backed-up devices where SharedPreferences is plain text and readable; use `flutter_secure_storage` (Keychain/Keystore).

C. It fails on network latency because SharedPreferences reads are synchronous and block the UI; use an in-memory cache.

D. It fails on multi-account support because SharedPreferences allows only one key per app; use a singleton.

26. scenario hard Evaluate

A backend team asks you to "just retry everything 3 times." For which single case is a retry both **safe and likely to help**?

A. A `GET /tasks` that failed with `connectTimeout`.

B. A `POST /tasks` that returned `422 Unprocessable Entity`.

C. A `DELETE /tasks/42` that returned `404 Not Found`.

D. A `PATCH /tasks/42` that returned `400 Bad Request`.

27. conceptual medium Apply

In `TaskRemoteDataSourceImpl.getTasks`, the catch maps `DioException` to `Unauthorized/Network/ServerException` and rethrows. Why is converting the error **here, at the data-source boundary**, correct?

A. So the repository and domain layers never depend on the `dio` package; the raw `DioException` is contained at the border and only typed app exceptions cross inward.

B. Because Dart cannot propagate a `DioException` across more than one `await` boundary, so it must be caught immediately.

C. So the UI can `catch (DioException)` directly and render an error widget without extra mapping.

D. Because `DioException` is a checked exception that the compiler forces you to handle at the call site.

28. code medium Analyze

A logging interceptor is placed after auth (assume a production build). The reviewer's most likely objection?

```
dio.interceptors.add(AuthInterceptor(() => readToken())); // adds Bearer
dio.interceptors.add(LogInterceptor(requestHeader: true)); // logs headers
```

A. Auth must come after logging, otherwise the token never reaches the server.

B. `LogInterceptor` runs after auth, so it logs the full `Authorization: Bearer <token>` header — leaking live tokens into logs unless stripped or disabled in release.

C. `requestHeader: true` causes the token to be sent twice, once in headers and once in the log channel to the server.

D. The order forces the token to be base64-decoded before logging, exposing the raw signing secret.

29. code · medium · Analyze

A new developer removes `handler.next(options)` from the real `AuthInterceptor.onRequest` and just lets the method return. What happens to every request?

A. Nothing changes; returning from `onRequest` implicitly continues the chain.

B. The request proceeds but skips all subsequent interceptors, reaching the network without retry or error handling.

C. The request hangs — without `handler.next`, the chain never advances, so the `Future` from `dio.get(...)` never completes.

D. Dio throws a synchronous `StateError` immediately on the missing handler call.

30. conceptual · medium · Understand

For a request the server answers with **HTTP 500**, when does `onResponse` fire versus `onError`?

A. `onError` fires, because by default Dio treats a non-2xx status as a failed request and routes it through the error path.

B. `onResponse` fires, because the server did return a response; `onError` only fires for thrown Dart exceptions.

C. Both fire in sequence: `onResponse` first to unwrap the body, then `onError` to map the code.

D. Neither fires; 5xx responses are dropped before reaching any interceptor.

31. scenario · hard · Evaluate

A `RetryInterceptor` retries a timed-out GET, which *succeeds* on the second attempt. For the design to behave correctly, what must be true about how retry and the error-mapping interceptor interact?

A. The error interceptor must run before retry on the request path so the typed exception is attached preemptively.

B. The error interceptor must map the first timeout to `NetworkException` and throw it, then the retry catches that exception and re-issues the call.

C. Both interceptors must run on every attempt, so the data source receives one `NetworkException` per failed try plus the final success.

D. The retry must fully resolve the successful attempt before the error interceptor maps anything, so a recovered request never surfaces as a typed error to the data source.

32. conceptual · medium · Apply

A `PUT /tasks/42` (full replace) is retried after a timeout; a `POST /tasks` (create) is not. Which property justifies treating them differently?

A. PUT is safe because it never reaches the database, while POST always commits immediately.

B. PUT is idempotent — replaying it with the same body yields the same final state — whereas POST creates a new resource each time, so replaying it can duplicate.

C. POST is idempotent but PUT is not, so the rule is actually backwards.

D. Both are idempotent; PUT is retried only because it is faster than POST on the wire.

33. code · medium · Analyze

A test wants to verify `getTasks()` throws `UnauthorizedException` on a 401 without hitting the real network. The data source is `TaskRemoteDataSourceImpl(this._dio)`. What makes this directly testable, and how?

A. The `Dio` instance is injected via the constructor, so a test can pass a mock Dio (or `http_mock_adapter`) that returns a 401 and assert the typed exception.

B. It is untestable as written, because `Dio` is constructed internally and cannot be replaced.

C. You must spin up a local HTTP server returning 401, since Dio cannot be mocked.

D. You override `getTasks()` in a subclass to return the exception, bypassing Dio entirely.

34. scenario · hard · Evaluate

`connectTimeout` and `receiveTimeout` are 15 s; `sendTimeout` is unset. A user on a weak uplink uploads a large attachment via POST; connection establishes, server is responsive, but the upload stalls partway. The failure mode?

A. `connectTimeout` will fire at 15 s and abort the stalled upload.

B. `receiveTimeout` will fire because no response bytes are arriving during the stall.

C. With no `sendTimeout`, the request-body upload can hang indefinitely, since connect and receive timeouts don't bound the send phase.

D. Dio falls back to `receiveTimeout` for the send phase when `sendTimeout` is null, so it aborts at 15 s.

Day 3 — DTOs, Serialization & Mapping

35. code · medium · Analyze

JSON arrives as `{"id": 5, "todo": "Buy milk", "dueDate": "2026-06-28"}` into a freezed `TaskDto` with `@JsonKey(name:'todo') String title`, `String? dueDate`. After `fromJson`, the runtime value and type of `dueDate`?

A. A `DateTime` for 2026-06-28, because json_serializable auto-parses ISO-8601 strings into DateTime.

B. A compile error, because a nullable `String?` cannot be deserialized without a `@Default`.

C. `null`, because `dueDate` has no `@JsonKey` so json_serializable cannot locate it in the JSON.

D. The `String` `"2026-06-28"`, kept literal, because the field is declared `String?` and no conversion happens until the mapper.

36. scenario · hard · Evaluate

Backend renames JSON `is_done` → `completed` overnight. Your DTO reads it via `@JsonKey(name:'is_done') bool completed`. Blast radius and the single correct fix?

A. The mapper breaks; fix the `toEntity()` line that reads `completed`.

B. Only the `@JsonKey(name: ...)` annotation in the DTO is wrong; change it to `'completed'` and regenerate — domain, mapper, and UI are untouched.

C. Every widget reading the field breaks; fix each widget's `['is_done']` access.

D. The entity's field name must change from `isDone` to `completed` to match the new API.

37. code · hard · Analyze

A developer "optimizes" the DTO: changes `dueDate` to `DateTime?` and adds `@JsonKey(fromJson: _toBool) bool completed` with `_toBool(v) => v=="1"||v==true`. JSON sends `dueDate` as `"28/06/2026"` (day-first) and done as `"1"`. Strongest senior critique?

A. The DTO now does interpretation (date format assumptions, `"1"` →bool) that belongs in the mapper; `DateTime` parsing of `"28/06/2026"` will fail/misparse and the DTO is no longer a literal mirror of the wire format.

B. It's correct and superior — converting in the DTO removes the mapper entirely.

C. `@JsonKey(fromJson:)` is invalid syntax; only `@JsonKey(name:)` is allowed.

D. The only problem is the missing `@Default(false)` on `completed`.

38. conceptual · medium · Understand

"Since `freezed` is annotated on `TaskDto`, I get `fromJson`/`toJson` for free — I don't need `json_serializable`." Evaluate.

A. Correct; `freezed` generates serialization on its own.

B. Correct, but only if you also delete the `.freezed.dart` file.

C. Incorrect; `freezed` gives immutability, `copyWith`, `==`, `toString`, but JSON serialization comes from `json_serializable` — you must add it and declare `fromJson`, which triggers the `.g.dart` generation.

D. Incorrect; `fromJson`/`toJson` can only be hand-written and never generated.

39. scenario · medium · Apply

You added a nullable field and ran `build_runner build`; it errors about an existing output (stale `task_dto.g.dart`). The correct, idiomatic resolution?

A. Hand-edit `task_dto.g.dart` to add the new field, then save.

B. Switch the annotation from `@freezed` to a plain class to skip generation.

C. Delete the `part 'task_dto.g.dart';` directive so the conflict disappears.

D. Re-run with `dart run build_runner build --delete-conflicting-outputs` so the generator overwrites the stale outputs.

40. code · hard · Analyze

The API contract says `description` is always present for active tasks but **omitted** for archived ones. A dev writes `required String description`. In production, archived tasks arrive without the key. What happens, and the modeling fix?

A. `fromJson` throws at runtime because a `required` non-nullable field is missing; model it honestly as `String?` (nullable) in the DTO and supply a default in the mapper if needed.

B. Nothing breaks; `required` fields default to empty string when absent.

C. The widget shows `null` silently; fix it in the UI with a null-check.

D. `build_runner` refuses to generate code for a sometimes-missing field, so it never compiles.

41. conceptual · medium · Evaluate

Why insist a `DateTime` conversion live in the **mapper** rather than the DTO, even though `json_serializable` *can* parse dates? Strongest justification?

A. Because `DateTime` cannot be a field type in a `freezed` class.

B. Because parsing (untyped→typed, done by the DTO) and interpretation (wire format→business meaning, done by the mapper) are distinct concerns; keeping the DTO a literal mirror lets one place absorb format quirks and API changes without touching the entity.

C. Because the mapper runs faster than json_serializable's converters.

D. Because `DateTime.parse` is only importable inside extension methods.

42. scenario hard Evaluate

During a 3-month migration, v1 sends `{"is_done":"1"}` and v2 sends `{"completed":true}`; the business meaning is unchanged. Cleanest design?

A. Add an `if (apiVersion == 2)` branch inside every UI widget that reads the field.
B. Change the `Task` entity to carry both `isDoneV1` and `isDoneV2` fields.
C. Create two DTOs (`TaskDtoV1`, `TaskDtoV2`), each literal to its wire format, and two mappers that both produce the *same* `Task` entity; the domain and UI never learn there were two versions.
D. Make the entity nullable everywhere so it tolerates either shape.

43. code hard Analyze

The real mapper line is `dueDate: dueDate == null ? null : DateTime.tryParse(dueDate!)`. The API sends `dueDate: "not-a-date"`. Resulting `Task.dueDate`, and why `tryParse` over `parse`?

A. Throws a `FormatException`; `tryParse` and `parse` behave identically on bad input.
B. An empty `DateTime()` at epoch zero, because `tryParse` never returns null.
C. The string `"not-a-date"`, because `tryParse` falls back to returning the original input.
D. `null`; `DateTime.tryParse` returns `null` on unparseable strings instead of throwing, so a malformed-but-present date degrades gracefully rather than crashing the app.

44. scenario medium Apply

A dev passes `TaskDto` straight into the task-list widget and reads `dto.completed`; it ships. Six weeks later the backend renames `todo` → `title`. The deeper architectural failure beyond the immediate breakage?

A. The DTO leaked into the UI, so API churn now propagates directly into widgets; the entity/mapper boundary that should have localized the change was bypassed.
B. None — using the DTO in the UI is fine as long as you keep `@JsonKey`.
C. The only failure is forgetting to run `build_runner watch`.
D. The bug is that `completed` should have been `required` instead of having a `@Default`.

45. code hard Analyze

A DTO has `required AuthorDto author` and `required List<TagDto> tags`. For this to deserialize correctly, what must be true of `AuthorDto` and `TagDto`?

A. Nothing extra; json_serializable can deserialize any class referenced by a DTO automatically.
B. Each must itself be a serializable type exposing a `fromJson` (e.g. its own `@freezed` + `json_serializable`), so the generator can recurse into the nested object and each list element.
C. `author` and `tags` must be marked `@JsonKey(ignore: true)` or generation fails.
D. The list must be `List<Map<String, dynamic>>` because json_serializable cannot build typed lists.

46. conceptual easy Understand

Passport analogy: DTO = foreign passport, entity = citizen ID, mapper = border. Which statement most faithfully extends it to a design rule?

A. The foreign passport (DTO) may travel freely throughout the country (domain/UI) as long as it's typed.
B. Citizens (entities) must carry the foreign passport (DTO) at all times for validation.
C. The border (mapper) issues the foreign passport, not the citizen ID.
D. Foreign passports (DTOs) are surrendered at the border (mapper) and never roam inside the country (domain/UI); only citizen IDs (entities) circulate internally.

47. code medium Analyze

A dev declares `required String title` but the JSON only sends `{"id":1,"todo":"Buy milk"}` — they forgot the annotation. Observable result?

A. `title` deserializes to `"Buy milk"` because json_serializable does fuzzy name matching.
B. Compiles and runs but `title` silently becomes `"todo"` (the key name).
C. `fromJson` throws or yields a missing-key error for `title`, because without `@JsonKey(name:'todo')` the generator looks for a JSON key literally named `title`, which isn't present.
D. `build_runner` auto-inserts the `@JsonKey` since the field is `required`.

48. scenario hard Evaluate

Crash: `type 'Null' is not a subtype of type 'String'` from `_$TaskDtoFromJson`, field `required String description`; the API occasionally returns `description: null`. Root **modeling** cause and fix?

A. The DTO dishonestly declared a sometimes-null field as non-nullable `required String`; model it as `String?` so parsing accepts null, then decide the business default in the mapper.
B. A mapper bug; wrap the mapper call in try/catch.
C. A `build_runner` caching issue; re-run with `--delete-conflicting-outputs`.
D. The entity's `description` should be `required`, forcing the API to comply.

49. conceptual medium Understand

Precise division of labor between `fromJson` and the mapper?

A. `fromJson` translates API language to business language; the mapper checks JSON shape.
B. `fromJson` parses (untyped JSON → typed DTO, catching shape errors at the border); the mapper translates (API-shaped DTO → business-shaped entity, decoupling from the API).
C. Both do the same thing; the mapper exists only for performance.
D. `fromJson` produces the entity directly; the mapper is optional cleanup.

50. code medium Apply

The real `toDto()` does `dueDate: dueDate?.toIso8601String()`. Why is that the correct symmetry with inbound `DateTime.tryParse(dueDate!)`, and what breaks with `dueDate.toString()` instead?

A. Nothing differs; `toString()` and `toIso8601String()` produce identical, round-trippable output.

B. Both are wrong; dates must be sent as Unix epoch integers.

C. `toString()` is correct and `toIso8601String()` would throw on null.

D. `toIso8601String()` emits the canonical ISO-8601 the DTO contract expects so it round-trips through `tryParse`; `DateTime.toString()` emits a space-separated format that isn't guaranteed to re-parse cleanly across the wire, and dropping `?.` would crash on a null `dueDate`.

51. scenario hard Evaluate

A dev hand-edits `task_dto.g.dart` to add a null check; tests pass; they commit. A week later a colleague runs `build_runner build` and the edit vanishes, reintroducing a bug. Principle violated and right approach?

A. Generated files (`.g.dart` / `.freezed.dart`) are overwritten on every regeneration and must never be hand-edited; the null-handling belonged in the DTO declaration (nullability/`@JsonKey`) or the mapper, so regeneration preserves it.

B. They violated nothing; editing `.g.dart` is fine if tests pass.

C. The fix was to add `.g.dart` to `.gitignore` so it can't be regenerated.

D. The error was running `build` instead of `watch`; `watch` would have kept the manual edit.

Day 4 — Repository & Data Sources

52. code · medium · Analyze

This repository method compiles and works in the happy path. The design defect a senior blocks on?

```
Future<Either<Failure, List<TaskDto>>> getTasks() async {
  try {
    final dtos = await _remote.getTasks();
    await _local.cacheTasks(dtos);
    return Right(dtos);
  } on NetworkException {
    return Left(const NetworkFailure());
  }
}
```

A. `cacheTasks` should be awaited only after the `Right` is returned so the UI is not blocked on disk I/O.

B. The return type is `Either<Failure, List<TaskDto>>` — the repository leaks DTOs into the domain instead of returning `List<Task>` entities.

C. It catches `NetworkException` but should let the exception propagate so the use case can decide on the fallback.

D. Caching before mapping is wrong; you must map to entities first and cache entities, not DTOs.

53. scenario · hard · Analyze

Two screens show the same task differently — "Today" says done, "All Tasks" says pending. Both read network-when-online, Hive-when-offline. Most likely root cause?

A. Hive returned stale data to one screen because its box was not flushed between writes.

B. The remote data source returned different DTOs because the two screens passed different cursors.

C. The repository mapped the DTO to an entity differently for each screen's use case.

D. Each screen implements its own `if (online) ... else ...` branching, so they diverge on when to trust the cache versus the network.

54. conceptual · medium · Understand

"We have a single source of truth — all our reads go through the one `TaskRemoteDataSource`." Why is this a misunderstanding of SSoT?

A. SSoT is about one *coordination point* (the repository) owning the data-access policy, not about having one data source.

B. SSoT refers to the data source being singular, but it must be the local source, not the remote one.

C. SSoT requires at least two data sources so the repository has something to coordinate.

D. It is correct only if the remote source also writes to the cache on every read.

55. code hard Evaluate

Review this remote data source; identify the most serious layering violation.

```
class TaskRemoteDataSourceImpl implements TaskRemoteDataSource {
  TaskRemoteDataSourceImpl(this._dio, this._local);
  final Dio _dio;
  final TaskLocalDataSource _local;

  Future<Either<Failure, List<TaskDto>>> getTasks() async {
    try {
      final res = await _dio.get('/todos');
      final dtos = parse(res);
      await _local.cacheTasks(dtos);
      return Right(dtos);
    } on DioException {
      return Left(const NetworkFailure());
    }
  }
}
```

A. It uses `Dio.get` without a typed response, which can throw at parse time outside the try.

B. It should cache *entities* rather than DTOs, so the mapping is missing before `cacheTasks`.

C. The remote source depends on the local source and returns `Either<Failure>` — it does coordination and error-policy work that belongs to the repository.

D. `NetworkFailure` is too generic; it should distinguish 401 from connection errors.

56. code medium Apply

Which option correctly realizes "network ok → cache + return fresh; network fails → return cached if any, else `NetworkFailure`"?

```
// Option A
try {
  final dtos = await _remote.getTasks();
  return Right(dtos.map((d) => d.toEntity()).toList());
} on NetworkException {
  return const Left(NetworkFailure());
}
```

```
// Option B
try {
  final dtos = await _remote.getTasks();
  await _local.cacheTasks(dtos);
  return Right(dtos.map((d) => d.toEntity()).toList());
} on NetworkException {
  final cached = await _local.getCachedTasks();
  return cached.isEmpty
      ? const Left(NetworkFailure())
      : Right(cached.map((d) => d.toEntity()).toList());
}
```

```
// Option C
final cached = await _local.getCachedTasks();
if (cached.isNotEmpty) return Right(cached.map((d) => d.toEntity()).toList());
final dtos = await _remote.getTasks();
return Right(dtos.map((d) => d.toEntity()).toList());
```

```
// Option D
try {
  final dtos = await _remote.getTasks();
  await _local.cacheTasks(dtos);
  return Right(dtos);
} on NetworkException {
  final cached = await _local.getCachedTasks();
  return Right(cached);
}
```

A. Option A

B. Option B

C. Option C

D. Option D

57. scenario · hard · Analyze

Network drops mid-fetch: Dio throws a `connectionError` after the request is sent but before a body arrives. With a correct read-through repository, what does the use case observe, and why?

A. `Right` with cached entities if the cache has data, else `Left(NetworkFailure)` — the remote source throws `NetworkException`, the repository catches it and falls back.

B. An unhandled `DioException` bubbles to the use case, because the failure happened inside the HTTP client, below the repository's catch.

C. `Left(ServerFailure)`, because a mid-fetch drop is a server-side condition.

D. `Right` with an empty list, because the partial response parses to zero tasks.

58. conceptual · medium · Understand

Why must Exception→Failure conversion happen in the repository rather than the data source?

A. Because `Failure` objects cannot be constructed inside the data layer for visibility reasons.

B. Because data sources are async and cannot use `try/catch` reliably.

C. Because the use case needs the original exception type to decide which `Failure` to emit.

D. Because the repository is the boundary to the domain; converting there keeps the domain free of raw exceptions while letting each source stay a dumb, throwing worker.

59. code · hard · Evaluate

A dev "fixes" error handling by having the data source return `Either<Exception, List<TaskDto>>` so the repository can drop try/catch. Why is this the wrong fix?

A. `Either<Exception, ...>` is invalid because `Exception` is not a sealed type and fpdart rejects it at compile time.

B. It is correct — returning `Either` everywhere is more functional and removes try/catch from the codebase.

C. The data source should throw, not return `Either`; pushing `Either` into the source duplicates error-handling responsibility and forces every source to share the repository's error vocabulary.

D. The problem is only that the left type should be `Failure`, not `Exception`; otherwise the design is sound.

60. scenario · medium · Apply

Unit-testing `getTasks` for the "remote fails, cache has data" path with mocktail. Which arrangement actually proves the fallback works?

A. Use a real Hive box seeded with data and a real Dio pointed at an unreachable host, then assert the result is `Right`.

B. Stub `remote.getTasks()` to throw `NetworkException`, stub `local.getCachedTasks()` to return DTOs, assert the result is `Right` with mapped entities, and verify `local.getCachedTasks` was called.

C. Stub `remote.getTasks()` to return DTOs and stub `local.getCachedTasks()` to throw, asserting `Left(NetworkFailure)`.

D. Stub only `local.getCachedTasks()` to return DTOs and leave `remote` unstubbed, asserting `Right`.

61. code · medium · Analyze

This `toggleDone` passed a junior's review. The correctness problem?

```
Future<Either<Failure, Task>> toggleDone(String id) async {
  try {
    final dto = await _remote.toggleDone(id);
    return Right(dto.toEntity());
  } on NetworkException {
    final cached = await _local.getCachedTasks();
    final match = cached.firstWhere((d) => d.id == id);
    return Right(match.toEntity());
  }
}
```

A. On offline fallback it returns the *old* cached state instead of reporting failure, so the UI shows the toggle as succeeded when it did not.

B. It should not catch `NetworkException`; toggling is a write and writes must always reach the server.

C. It maps DTO→Entity, which a write method must not do; writes return `Unit`.

D. `firstWhere` is fine; the only issue is the missing `cursor` parameter.

62. conceptual · hard · Evaluate

A senior argues caching/branching logic "must not live in the UI." Beyond DRY, the strongest architectural justification?

A. Widgets rebuild frequently, so putting `if (online)` in `build` would re-run the network check on every frame.

B. The UI cannot import data-layer types, so it is technically impossible to branch there.

C. Centralizing the rule in the repository makes the data-access policy independently testable and guarantees every consumer observes identical behavior, eliminating cross-screen contradictions.

D. Branching in the UI breaks hot reload because state would not survive a reload.

63. scenario medium Analyze

After a refactor, `getTasks` always returns fresh server data when online but feels slow on 3G and occasionally shows a blank list for a second before data appears. The offline-fallback branch is correct. Most likely remaining gap vs. read-through design?

A. `NetworkFailure` is being returned even when the network succeeds.
B. The repository is mapping DTO→Entity on the main isolate, which blocks the UI.
C. The cache is being read before the network on every call, slowing the happy path.
D. The repository is not caching successful responses, so each call pays full network latency and there is nothing to show instantly while the network is in flight.

64. conceptual medium Understand

Where does DTO→Entity mapping belong, and what guarantees that choice?

A. In the data source, so the source returns ready-to-use entities and the repository stays thin.
B. In the repository, because it is the data/domain boundary; mapping there keeps DTOs from leaking into the domain while sources stay focused on I/O.
C. In the use case, so the domain decides how to interpret each field.
D. In the widget, mapped lazily as fields are displayed, to avoid wasted conversions.

65. code hard Analyze

You're adding the Day 4 local source to the current `getTasks` (which maps exceptions to `Unauthorized/Network/ServerFailure`). Which change preserves the existing exception-mapping contract **while** adding read-through caching?

A. After the successful `Right`, also `await _local.cacheTasks(dtos)` before returning, and in the `NetworkException` catch, read the cache and return cached entities if present (else keep `NetworkFailure`).
B. Move the `try` into the `NetworkException` catch so the cache is read first, then fall through to the network.
C. Add the local read inside the `UnauthorizedException` and `ServerException` catches as well, so every failure falls back to cache.
D. Wrap the whole method so it returns the cache first and only calls the network when `getCachedTasks` is empty.

66. scenario · medium · Apply

New requirement: on every cache write, the local source should also call the remote source's analytics-logging endpoint. A senior rejects wiring this into `TaskLocalDataSourceImpl`. Why?

A. Analytics calls are too slow to run inside a Hive transaction.

B. It is fine architecturally; the only concern is retry logic for the analytics call.

C. The local source would then import and depend on the remote source, violating the rule that a data source must not know about the other; coordination across sources belongs in the repository.

D. Hive cannot perform network calls, so it is technically impossible.

67. conceptual · hard · Evaluate

Which symptom most precisely identifies a "god-repository"?

A. The repository returns `Either<Failure, T>` from every method.

B. The repository contains the parsing, HTTP, and DB-access code itself instead of delegating to narrow data sources it coordinates.

C. The repository injects both a remote and a local data source.

D. The repository maps DTOs to entities for several different methods.

68. code · hard · Evaluate

The author claims this test proves the repository "caches on success." Does it, and what is the gap?

```
test('returns tasks on success', () async {
  when(() => remote.getTasks(cursor: any(named: 'cursor')))
      .thenAnswer((_) async => [taskDto]);
  when(() => local.cacheTasks(any())).thenAnswer((_) async {});
  final result = await repo.getTasks();
  expect(result.isRight(), true);
});
```

A. Yes — stubbing `local.cacheTasks` and getting a `Right` is sufficient proof that the cache write occurred.

B. Yes for caching, but it wrongly stubs `remote` with a list instead of a single DTO.

C. No — the test should use a real Hive box, because mocking `cacheTasks` makes the caching assertion meaningless by definition.

D. No — it never asserts `verify(() => local.cacheTasks(any())).called(1)`, so the test passes even if the repository never caches; it also never checks the mapped entity value.

Day 5 — Caching, Offline-First & Pagination

69. conceptual · medium · Evaluate

The profile screen shows name/avatar/tier — values that change a few times a year — and must open instantly while tolerating brief staleness. Best strategy and why?

A. Network-first, because profile data must always be guaranteed fresh before display.

B. Write-through, because reading the profile is fundamentally a write operation that must hit both layers.

C. Cache-first, because rarely-changing data can be served instantly from cache and only fetched on a miss.

D. Stale-while-revalidate, because every read must trigger a background network refresh regardless of how rarely the data changes.

70. code · medium · Analyze

The defect in this TTL check?

```
bool isFresh(DateTime cachedAt, Duration ttl) {
  return DateTime.now().difference(cachedAt) > ttl;
}
// usage:
final cache = await local.readTasks();
if (cache != null && isFresh(cache.cachedAt, ttl)) {
  return cache.data; // serve cache
}
return await fetchFromNetwork();
```

A. The comparison is inverted: `difference > ttl` means the cache is *stale*, so fresh data is treated as expired and stale data is served.

B. `DateTime.now()` should be in UTC, otherwise the TTL is always negative.

C. `isFresh` should compare against `cachedAt` plus `ttl` using `isBefore`, which is the only correct way and the current form cannot work.

D. The network fetch should happen before the cache read so the cache is always warm.

71. scenario · hard · Analyze

A user creates five tasks offline; each saves to local DB, updates the UI, and pushes onto an **in-memory** `List<SyncAction>`. They force-close the app before signal returns. Later they reopen with signal: tasks are visible locally but never reach the server. Root cause?

A. Optimistic UI updates are inherently unsafe and should never be used for offline writes.

B. Last-write-wins conflict resolution silently discarded the five tasks on reconnect.

C. The local DB write should have been skipped until the network confirmed each task.

D. The sync queue was held in memory, so it was lost on app close; it must be persisted to disk to survive a restart.

72. scenario · medium · Analyze

While scrolling a long list, a user sees the same task twice as pages load. Backend uses `?offset=&limit=`; other users frequently add tasks to the top in real time. Most likely root cause?

A. The `hasMore` flag is never set to false, causing the list to loop back to the start.
B. Offset pagination is being used on a list with frequent inserts at the top, so each insert shifts every row down and a page boundary re-serves an already-seen row.
C. The pages are being rebuilt instead of appended, which duplicates earlier items.
D. The TTL on the cached pages expired mid-scroll, forcing a re-parse that doubles entries.

73. code · hard · Analyze

Two reviewers flag problems in this infinite-scroll handler. Which **pair** of defects is real?

```
Future<void> loadMore() async {
  final page = await repo.getTasks(cursor: _cursor);
  _items = page.items;          // line A
  _cursor = page.nextCursor;
}
// onScroll: if (nearBottom) loadMore();
```

A. The cursor is updated too early, AND pages must always be fetched on the UI thread.
B. `cursor` should be an offset integer, AND `getTasks` should be synchronous.
C. Line A rebuilds the list instead of appending (`_items = page.items` discards prior pages), AND there is no `isLoadingMore` guard, so a fast scroll fires overlapping `loadMore` calls.
D. `nearBottom` should trigger on every pixel of scroll, AND the list must refetch page 1 each time.

74. conceptual · hard · Evaluate

A screen shows the user's wallet balance, which can change from server-side transactions the device never initiated; the team wants correctness over instant render. Right default strategy?

A. Stale-while-revalidate, so the balance renders instantly and corrects itself a moment later.
B. Cache-first, because any cached balance is good enough to display immediately.
C. Write-through, because reading a balance should also write it back to both cache and server.
D. Network-first, because the balance must be current and the cache should only be a fallback when the network fails.

75. scenario · medium · Apply

A reviewer worries SWR will "freeze on first paint while it revalidates." Which statement correctly describes SWR?

A. SWR serves the cached data immediately and triggers the network refresh in the background; the first paint does not block on the network.
B. SWR blocks rendering until the background revalidation completes, then paints once with fresh data.
C. SWR skips the cache entirely on the first read and only uses it on subsequent reads.
D. SWR writes to the network first and only then reads from cache, so the first paint waits on a round trip.

76. conceptual · hard · Evaluate

Two teammates edit the same task's free-text `notes`, one offline. On reconnect they conflict. The PO says "never silently lose either person's typing." Best approach and its cost?

A. Last-write-wins — cheap and deterministic, and it satisfies the constraint because the latest timestamp is always the most correct.
B. Field-level merge (or prompting the user) — it can preserve both contributions, at the cost of significant implementation complexity.
C. Server-wins — safe and simple, and it satisfies the constraint because the server copy is authoritative.
D. Drop the offline edit before sync — simplest, and acceptable because offline edits are inherently lower priority.

77. code · hard · Analyze

The list has 47 items. After the last page loads, the user keeps scrolling at the bottom and the app keeps firing requests that return empty lists. What is missing?

```
Future<void> loadMore() async {
  if (_isLoadingMore) return;
  _isLoadingMore = true;
  final page = await repo.getTasks(cursor: _cursor);
  _items = [..._items, ...page.items];
  _cursor = page.nextCursor;
  _isLoadingMore = false;
}
```

A. A switch from cursor to offset paging so the end of the list is detectable.
B. The `_isLoadingMore` guard, which is already absent here.
C. A larger page size so 47 items fit in one page.
D. A `hasMore` flag: once a page returns empty (or `nextCursor` is null), `hasMore` must be set false and `loadMore` must early-return on it.

78. scenario · medium · Apply

In a tunnel, a user toggles a task to "done." The app must feel instant and the change must reach the server eventually. Which sequence implements offline-first correctly?

A. Update the UI optimistically but skip local persistence, relying on the in-memory state to survive until signal returns.

B. Show a spinner, attempt the server call, and only update local state after the server responds.

C. Write to local DB and update the UI immediately, enqueue a persisted sync action, and flush the queue to the server when connectivity returns.

D. Block the toggle entirely until connectivity is detected to avoid any conflict.

79. conceptual · medium · Understand

Why does a cursor stay correct when offset/page-number paging drifts?

A. A cursor points at a specific item ("give me items after this one"), so inserts or deletes elsewhere don't shift what comes next, whereas a numeric offset shifts when the list changes.

B. A cursor encodes an absolute row index, so the server can always recompute the exact numeric position.

C. A cursor caches the entire result set on the client, so the server is never re-queried.

D. A cursor forces the list to be immutable on the server for the duration of the scroll.

80. scenario · hard · Analyze

After scrolling 6 pages (120 tasks), pull-to-refresh re-fetches all 6 pages sequentially, freezing for seconds before showing page 1. What should refresh do instead?

A. Re-fetch all 6 pages but on a background isolate so the freeze is hidden from the user.

B. Reset to the first page (cursor cleared) and fetch only page 1, caching it, rather than re-fetching every previously loaded page.

C. Keep the existing 120 items and append a fresh page 7, since refresh means "load newer data."

D. Clear the local cache entirely and show a spinner until all pages reload, guaranteeing freshness.

81. code · medium · Analyze

Given the real remote `getTasks` (reads `res.data?['todos']`, passes `cursor` as a query param) and a repository that maps to entities, what must **additionally** flow back from this layer to make correct cursor-based infinite scroll?

A. The numeric offset of the last item, so the next call can pass `offset`.

B. Nothing — returning the list of DTOs is sufficient for cursor paging.

C. The total row count, which is mandatory for cursor paging to function.

D. A `nextCursor` (and/or a `hasMore` signal) from the response, so the caller knows what to request next and when to stop; right now only the items are returned.

82. scenario · medium · Apply

A teammate sets up a list cache with no TTL "to keep it simple." Days later, tasks deleted on another device still show and never disappear. Underlying problem?

A. The cache lacks an expiry, so once written it is served indefinitely and never revalidated against the network.
B. The cache is using stale-while-revalidate, which by design never refreshes.
C. The deletions failed because cursor pagination cannot represent a deleted row.
D. The cache is network-first, so it ignores local writes.

83. conceptual · hard · Evaluate

"Offline-first gives us strong consistency because the local DB is the single source of truth." Which correction is accurate?

A. Correct as stated — local-as-primary means the system is strongly consistent.
B. Offline-first has no consistency model; data correctness is not a concern in offline apps.
C. Offline-first yields *eventual* consistency: local and server states can diverge temporarily and only converge after the queue syncs, which is why conflict resolution exists.
D. Offline-first guarantees the server is always ahead of the client, so the client is never authoritative.

84. scenario · hard · Analyze

Cursor paging works and pages append. A user creates a task offline (saved locally + enqueued); it appears at the top immediately. After sync assigns it a server ID/position, scrolling down reveals the **same** task a second time. Cause and fix?

A. Cursor pagination is broken; switch to offset to deduplicate.
B. The locally-created (optimistic) item and the server-returned item are distinct entries; on sync the local temp entry must be reconciled/replaced by ID rather than left in place alongside the synced copy.
C. The TTL expired, so the task was cached twice.
D. `hasMore` was never set, so the list looped and re-showed the task.

Day 6 — Error Handling & Authentication

85. conceptual · medium · Understand

Why must Exception→Failure translation happen at the **repository** and not later (UseCase/UI)?

A. The UseCase and UI depend only on the domain contract, which speaks `Failure`; if a raw `Exception` escaped the repository, it would be an untyped surprise the compiler never forced anyone to handle.

B. The repository runs on a background isolate, so it's the only layer allowed to catch exceptions without freezing the UI thread.

C. UseCases cannot contain `try/catch` blocks in clean architecture, so catching anywhere else is a compile error.

D. Translating later is fine functionally, but the repository is conventionally chosen for readability; any layer could legally catch the `DioException`.

86. code · hard · Analyze

The data source can also throw a raw `DioException` (a timeout never wrapped into a typed exception). For the `getTasks` below, what actually happens, and why is it the dangerous case?

```
try {
  final dtos = await _remote.getTasks(cursor: cursor);
  return Right(dtos.map((d) => d.toEntity()).toList());
} on UnauthorizedException {
  return const Left(UnauthorizedFailure());
} on NetworkException {
  return const Left(NetworkFailure());
} on ServerException catch (e) {
  return Left(ServerFailure(e.message));
}
```

A. The `DioException` is silently swallowed and the method returns `Right` with an empty list, hiding the error.

B. Dart automatically maps any unmatched exception to the last `catch` clause, so it becomes a `ServerFailure`.

C. The `DioException` matches no `on` clause, so it propagates out of the repository as an uncaught exception — exactly the leak `Either` was meant to prevent.

D. The `await` rethrows it as a `NetworkException`, so it's safely caught by the network clause.

87. conceptual easy Understand

A colleague writes `result.fold((x) => x, (y) => showError(y))` on an `Either<Failure, T>`. What's wrong with this mental model?

A. Nothing — `fold` takes success first, then failure.

B. `Left` holds the failure and `Right` holds the success; they have swapped the two callbacks, so success values get returned raw and failures get passed to `showError`.

C. `fold` only accepts a single callback, so the code will not compile regardless of order.

D. `Right` is the failure side because it "writes" the error, so the callbacks are actually correct.

88. scenario medium Apply

A user taps "refresh" in a no-signal tunnel. With the failure pipeline correctly built, what does the user see, and which type carries the info at the UI?

A. A red error overlay with a `DioException` stack trace, because network errors cannot be translated.

B. A generic "Something went wrong" message, because all offline errors collapse into `ServerFailure` by design.

C. The app crashes to the OS, since an offline device cannot run the fold logic.

D. A friendly "No internet connection." message, because the data source threw `NetworkException`, the repository returned `Left(NetworkFailure())`, and the UI folded it to that message.

89. conceptual medium Analyze

Why two tokens (access + refresh) instead of one long-lived token sent on every request?

A. The access token is exposed on every request so it is short-lived (a stolen one expires fast); the refresh token is rarely transmitted so it can be long-lived and minting-only — limiting blast radius if the access token leaks.

B. Two tokens let the server load-balance requests across auth servers; it is a scaling decision, not a security one.

C. The refresh token is sent on every request and the access token only at login, splitting the work evenly.

D. One token would work identically; the second exists only because Dio's interceptor API requires a separate refresh token field.

90. scenario hard Evaluate

A dev stores the access token in `flutter_secure_storage` but keeps the refresh token in `SharedPreferences` "because it's just a string." Why is this the more serious of the two to get wrong?

A. It is harmless — `SharedPreferences` is encrypted by default on both platforms.

B. The refresh token is the long-lived, high-value credential; leaking it from plaintext `SharedPreferences` lets an attacker mint fresh access tokens for as long as it lives — a far bigger blast radius than a short-lived access token.

C. `SharedPreferences` cannot store strings, so the refresh flow will fail to compile.

D. Both tokens are equally low-risk in `SharedPreferences` since they expire; storage location is irrelevant once expiry exists.

91. code hard Analyze

This interceptor has no guard flag. The specific failure mode when the **refresh token itself** has expired?

```
void onError(DioException err, ErrorInterceptorHandler handler) async {
  if (err.response?.statusCode == 401) {
    final newToken = await _refresh();        // calls POST /refresh
    final req = err.requestOptions;
    req.headers['Authorization'] = 'Bearer $newToken';
    handler.resolve(await _dio.fetch(req));
    return;
  }
  handler.next(err);
}
```

A. The original request silently succeeds with the expired token because `fetch` ignores 401s.

B. Nothing bad — Dio detects recursion automatically and breaks the loop after one retry.

C. `POST /refresh` returns 401, which re-enters `onError`, which calls `_refresh()` again, which 401s again — an infinite refresh loop that never terminates or logs the user out.

D. The interceptor throws a `StackOverflowError` immediately on the first 401, before any network call.

92. scenario hard Evaluate

Ten queued requests all hit a 401 within the same instant because the access token just expired; a naive interceptor fires a refresh for each. Correct senior-level design?

A. Fire all ten refreshes in parallel; whichever new token returns last wins, ensuring freshness.

B. Let each request refresh independently; the server deduplicates refresh tokens so it is harmless.

C. Reject nine of the ten requests with `ServerFailure` and only let the first succeed, to avoid duplication.

D. Detect that a refresh is already in flight, queue the other nine requests, perform a *single* refresh, then replay all ten with the new token.

93. scenario medium Apply

A user has been away two weeks; their refresh token has expired. The next call 401s, refresh fails. Correct UX per the lesson?

A. Hard logout: clear the stored tokens, reset auth state to logged-out, and redirect to login — ideally remembering where they were so they return after re-auth.

B. Show a red `UnauthorizedException` dialog with a "retry" button that re-calls the same failing endpoint.

C. Silently keep retrying the refresh every few seconds in the background until a new session somehow appears.

D. Keep the user on the current screen but disable all buttons, leaving the stale token in storage.

94. code medium Apply

A teammate adds `final String accessToken;` to the `User` entity (in `props` too). Why is putting `accessToken` on the entity a design mistake?

A. `Equatable` cannot include token strings in `props`, so the entity will not compile.

B. It is fine and recommended, since the UI needs the token to authorize requests directly.

C. The `User` entity is a domain identity object; tokens are transient auth credentials that belong in secure storage, not coupled to who the user *is* — mixing them leaks a credential into every place a `User` is passed.

D. Tokens must live on the entity so `Equatable` can detect when a session changes.

95. scenario medium Apply

During login a dev stores the user's email and **password** in secure storage "so we can silently re-login when the refresh token dies." Critique.

A. Reasonable — secure storage is encrypted, so storing the password there is equivalent to storing a token.

B. Wrong — you never store passwords anywhere; the refresh token exists precisely so the long-lived secret is a revocable, minting-only token, not the user's reusable master credential.

C. Fine, as long as the password is base64-encoded before storage.

D. Acceptable only if the password is also kept in `SharedPreferences` as a backup.

96. conceptual hard Evaluate

Why does `Either<Failure, T>` "beat throwing," beyond style?

A. `Either` is faster at runtime because it avoids the cost of building stack traces.

B. Throwing is illegal in Dart's null-safe mode, so `Either` is the only option that compiles.

C. `Either` cannot represent success, so it eliminates the ambiguity of partial results.

D. `Either` turns the failure into a value the type system forces the caller to handle (both `Left` and `Right`), whereas a thrown exception relies on the caller *remembering* to catch — a silent omission that crashes at runtime.

97. code medium Analyze

The real `AuthInterceptor` attaches the token on `onRequest` but has no `onError`. Relative to the lesson's silent-refresh design, the most accurate assessment?

A. It only handles the *happy path*; with no `onError` to catch 401s, an expired access token surfaces as a failure to the user instead of being silently refreshed and retried.

B. It is complete; attaching the token on `onRequest` is all the silent refresh requires.

C. It is broken because `onRequest` should attach the *refresh* token, not the access token.

D. It will loop infinitely because `onRequest` re-runs on every retry.

98. scenario hard Analyze

An interceptor sets `isRefreshing = true` but **never resets it to false** after a successful refresh. Symptom users report, and root cause?

A. Symptom: tokens leak to the UI — root cause is missing exception translation.

B. Symptom: the very first 401 after a successful refresh is never refreshed again, so every later session expiry fails permanently — root cause is the stuck guard flag blocking all future refresh attempts.

C. Symptom: login fails immediately — root cause is the access token never being attached.

D. Symptom: passwords get stored — root cause is the guard flag being a boolean.

99. conceptual medium Understand

In the journey API→DataSource→Repository→UseCase→UI, at which two points do the error's **type and form** fundamentally change?

A. The UseCase throws the exception, and the UI catches it with `try/catch`.

B. The API converts to a Failure, and the UI converts it back to an exception for logging.

C. The DataSource throws an exception, and the Repository converts it into a returned `Left(Failure)` value; from there it stays a `Failure` value through UseCase and UI.

D. Every layer re-throws a fresh exception, so the type changes at all five hops.

100. conceptual hard Evaluate

A team maps **every** data-layer exception to a single `ServerFailure("Something went wrong")` to "keep the UI simple." Which critique aligns with the lesson?

A. Correct approach — one failure type is cleaner and the user does not care about the difference.

B. It is fine for network errors but server errors must be re-thrown as raw exceptions to the UI.

C. It is wrong because every exception should instead map to `ValidationFailure`.

D. It collapses distinct, actionable conditions (no internet vs expired session vs server error) into one message, so the UI cannot show "No internet connection." or trigger a re-login — the "generic message for everything" trap.